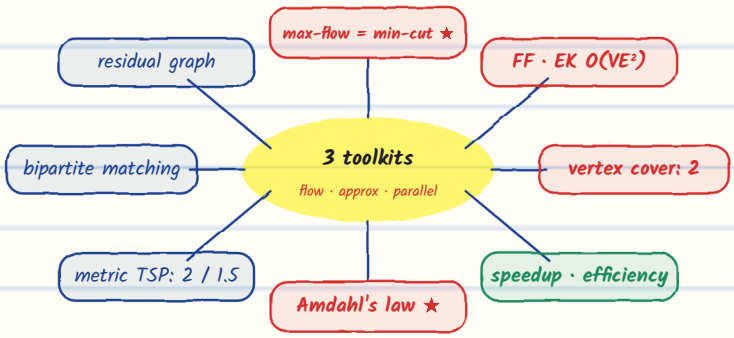
Flow, Approximation & Parallel Algorithms

max-flow = min-cut · Ford–Fulkerson · Edmonds–Karp · ρ-approximation · PTAS/FPTAS · speedup · Amdahl

Eleven sheets: how much can a network carry and where it chokes, how to get a provably good answer when the exact one is out of reach, and how fast many processors really make a program — the three "beyond the basics" clauses of the algorithms syllabus.



↑ the whole topic: red = high-yield, green = the parallel metrics

➜ Syllabus coverage

The algorithms syllabus lists, after graph traversal, spanning trees and shortest paths: "Flow algorithms; Approximation algorithms; Parallel algorithms;" — followed by lower-bound theory and NP-completeness. **All three clauses are taken in full here.** Graph searching (BFS/DFS), MST and greedy methods come from the previous lesson; why some problems need approximation at all — NP-hardness — is the next lesson. Here the question is practical: how much, how close, how fast?

High-yield list

① $0 \le f \le c$; in = out except at s, t
② max flow = min cut
③ Residual: forward $c-f$, backward f
④ FF $O(E \cdot |f^*|)$, integer capacities
⑤ Edmonds–Karp = BFS paths, $O(VE^2)$
⑥ Dinic $O(V^2E)$ · push-relabel $O(V^2E)$
⑦ Matching = flow with unit capacities
⑧ $\rho \ge 1$; vertex cover 2-approx
⑨ Metric TSP 2 (MST), Christofides 1.5
⑩ Set cover greedy $H(n) \approx \ln n$
⑪ $S = T_1/T_p$, $E = S/p$
⑫ Amdahl: $S \le 1/((1-P)+P/N)$

Legend yellow = key term · red circle = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

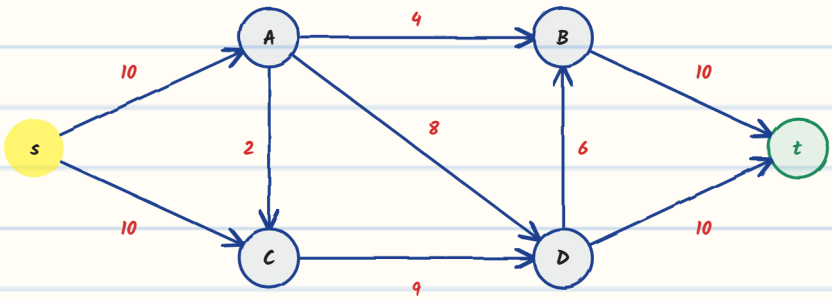
➜ Contents

✓ Flow networks & residual graph … p. 2
✓ Ford–Fulkerson & complexities … p. 3
✓ Worked max-flow + min cut … p. 4
✓ Max-flow min-cut & applications … p. 5
✓ Approximation: ratio, vertex cover … p. 6
✓ TSP, set cover, knapsack, PTAS … p. 7
✓ Parallel metrics & Amdahl's law … p. 8
✓ Parallel sorting, matrix, BFS … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 Flow Networks: Source, Sink, Capacity, Flow — and the Residual Graph

the vocabulary ★

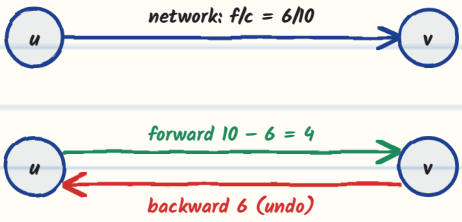
Definition A **flow network** is a directed graph $G = (V, E)$ in which every edge (u, v) has a capacity $c(u, v) \geq 0$, with two special vertices: the **source** s, where flow originates, and the **sink** t, where it is absorbed. A **flow** f assigns to each edge the amount actually passing through it. Water pipes, road lanes, network links and conveyor belts are all the same model: how much can travel from s to t per unit time?



the running example of this lesson: red numbers are capacities; it is solved on sheet 4

Rule	Statement	Meaning
Capacity constraint	$0 \leq f(u,v) \leq c(u,v)$	No edge carries more than its capacity, and never a negative amount.
Flow conservation	$\sum f(\cdot, v) = \sum f(v, \cdot)$ for every $v \neq s, t$	What enters a vertex leaves it — only s and t are exempt.
Value of the flow	$\lvert f \rvert = \sum f(s, v) - \sum f(v, s)$	Net flow out of the source = net flow into the sink.
Bottleneck	$c_f(p)$ = min residual capacity on path p	The amount one augmenting path can add.

Residual graph G_f ✍ For every edge carrying f of capacity c, the residual graph has a **forward edge** with $c_f(u,v) = c(u,v) - f(u,v)$ (room left) and a **backward edge** with $c_f(v,u) = f(u,v)$ (flow that may be cancelled). An **augmenting path** is any $s \to t$ path in G_f; edges with $c_f = 0$ are dropped.



one edge ⟶ two residual edges

Exam trap ✍ three **1.** Conservation does **not** hold at s and t — that is where the flow value is measured. **2.** The backward edge is not a real pipe; pushing along it means cancelling flow already sent. **3.** A saturated edge ($f = c$) disappears forward but **still exists backward**.

Memory trick "**Forward = room left, backward = regret.**" Every unit you send buys you the right to change your mind about it later — that is the residual graph in one line.

2 Ford–Fulkerson, Edmonds–Karp, Dinic and Push–Relabel

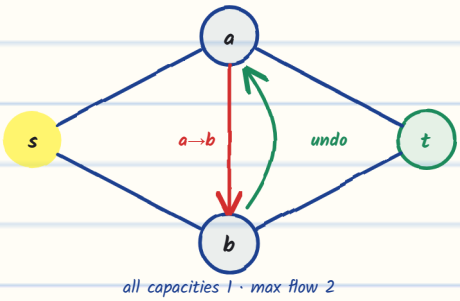
the complexity table ★

```
FORD-FULKERSON(G, s, t)
  for each edge (u,v): f(u,v) = 0
  while an s→t path p exists in G_f:
    b = min{ c_f(u,v) : (u,v) on p }
    for each (u,v) on p:
      if (u,v) ∈ E: f(u,v) += b
      else:         f(v,u) -= b
  return f        // |f| is maximum
```

The method in four steps 1. Start with zero flow. 2. Find any augmenting path in the residual graph. 3. Push its **bottleneck** b along it and update residual capacities (forward $-b$, backward $+b$). 4. Repeat until no $s\to t$ path remains; the flow is then maximum. FF is a *method*: it does not say **how** to find the path.

Algorithm	How the path / flow is chosen	Time
Ford–Fulkerson (1956)	Any augmenting path (often DFS). Needs **integer** capacities to be sure of stopping.	$O(E\cdot\lvert f^*\rvert)$
Edmonds–Karp (1972)	FF with BFS = shortest augmenting path (fewest edges). $O(VE)$ augmentations × $O(E)$ per BFS.	$O(VE^2)$
Dinic (1970)	BFS builds a **level graph**; DFS pushes a **blocking flow**; at most $V-1$ phases.	$O(V^2E)$; unit capacity $O(E\cdot\min(V^{2/3}, E^{1/2}))$
Push–Relabel (Goldberg–Tarjan)	Not path-based: keeps a **preflow**, pushes excess downhill, **relabels** heights.	generic $O(V^2E)$; FIFO $O(V^3)$
Hopcroft–Karp	Dinic specialised to **bipartite matching**.	$O(E\sqrt{V})$

Exam trap ✍ three corrections 1. Dinic is $O(V^2E)$ in general. The bound $O(E\cdot\min(V^{2/3}, E^{1/2}))$ holds only for unit-capacity networks — quoting it as the general bound is wrong. 2. $O(E\cdot\lvert f^*\rvert)$ assumes integer capacities: each augmentation adds ≥ 1. With irrational capacities plain FF may **never terminate** (and may even converge to a wrong value). 3. Edmonds–Karp's bound is independent of the capacities — that is exactly why BFS is used.



all capacities 1 · max flow 2

$s\to a\to b\to t$ first: only the backward edge $b\to a$ lets $s\to b\to a\to t$ reach flow 2

Memory trick "**Ford finds any road, Edmonds–Karp takes the shortest, Dinic floods by levels, push–relabel lets water run downhill.**" Complexities pair up: **E–K = VE^2**, **Dinic = V^2E** — the squares swap places.

Integrality theorem With **integer** capacities, FF returns a max flow in which every edge carries an integer. This is what makes the matching reduction (sheet 5) give a real matching, not fractions.

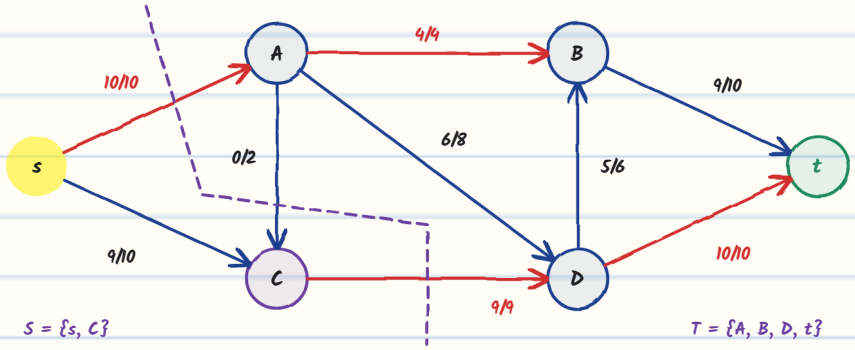
Worked example ✍ why the path choice matters $s\to u$ 1000, $s\to v$ 1000, $u\to v$ 1, $u\to t$ 1000, $v\to t$ 1000. A careless FF can alternate $s\to u\to v\to t$ and $s\to v\to u\to t$ (through the backward edge $v\to u$), adding **1** each time: 2000 augmentations. Edmonds–Karp picks the 2-edge paths $s\to u\to t$ and $s\to v\to t$ first: **2 augmentations**, same max flow **2000**.

3 Worked Example: Max Flow of the Sheet-2 Network, Checked by a Min Cut

written-exam favourite ★

Worked example ✍ Edmonds-Karp (shortest paths first) Network: s→A 10, s→C 10, A→B 4, A→C 2, A→D 8, C→D 9, D→B 6, B→t 10, D→t 10. Each row: the path found in the current residual graph, its bottleneck, and what it saturates.

#	Augmenting path	Residual capacities on the path	Bottleneck	Saturates	|f|
1	s→A→B→t	10, 4, 10	4	A→B	4
2	s→A→D→t	6, 8, 10	6	s→A	10
3	s→C→D→t	10, 9, 4	4	D→t	14
4	s→C→D→B→t	6, 5, 6, 6	5	C→D	19
5	From s only C is reachable (s→A = 0, C→D = 0, no flow on A→C to cancel) — **no augmenting path: stop.**				



final flow f/c (red = saturated); the purple dashed line is the minimum cut

Cut S | T	Forward edges S→T	c(S,T)
{s}	10 + 10	20
{s, A, C}	4 + 8 + 9	21
{s,A,B,C,D}	10 + 10	20
{s, C}	s→A 10 + C→D 9	19

Check every vertex A: in 10, out 4+0+6 = 10 ✓ · C: in 9, out 9 ✓ · D: in 6+9 = 15, out 5+10 = 15 ✓ · B: in 4+5 = 9, out 9 ✓ · out of s 10+9 = **19** = into t 9+10 ✓. The vertices reachable from s in the final residual graph form the min-cut side S.

Exam trap ✍ two **1.** A→C crosses the cut from T to S, so it is not added to the cut capacity — a cut counts only **S→T** edges. **2.** The obvious cuts ({s} = 20, into t = 20) are **not** the minimum; always search the residual graph for the true S.

Reading the answer like an engineer The min cut names the **bottleneck links**: only raising **s→A** or **C→D** can increase the throughput. Upgrading B→t or D→t to 100 changes nothing, because the flow is limited by the cut, not by the sink side. Edges outside every min cut have slack.

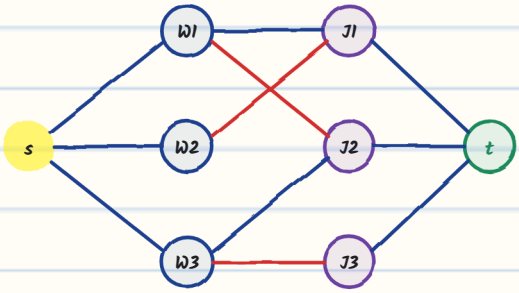
4 The Max-Flow Min-Cut Theorem, and Where Flow Is Used

one theorem, many problems ★

Definitions An **s–t cut** (S, T) splits V into two parts with $s \in S$ and $t \in T$. Its **capacity** is $c(S,T) = \Sigma\, c(u,v)$ over edges $u \in S$, $v \in T$ — forward edges only. The **net flow across** it is forward flow minus backward flow, and for every cut it equals $|f|$.

Max-flow min-cut theorem (Ford–Fulkerson, 1956) The following are **equivalent**: **(1)** f is a maximum flow; **(2)** the residual graph G_f has no augmenting path; **(3)** $|f| = c(S, T)$ for some cut. Hence **max $|f|$ = min $c(S,T)$**. The easy half (**weak duality**): every flow ≤ every cut, because all flow must cross the cut.

Bipartite matching as a flow Left set L, right set R. Add $s \to$ **every** $u \in L$ and **every** $v \in R \to t$, orient each L–R edge left to right, and give every edge capacity 1. By the integrality theorem the max flow is 0/1 on each edge, and the edges $L \to R$ carrying flow form a **maximum matching**: **max matching = max flow**.



red = matched (cap 1 each)

matching = unit-capacity flow

Worked example Workers W1, W2, W3; jobs J1, J2, J3. W1 can do J1, J2; W2 only J1; W3 J2, J3. Paths $s \to W1 \to J1 \to t$, $s \to W3 \to J2 \to t$ give 2; then $s \to W2 \to J1$ is blocked forward, but the residual edge **J1→W1** lets $s \to W2 \to J1 \to W1 \to J2 \to W3 \to J3 \to t$ re-route: flow **3** = W1–J2, W2–J1, W3–J3.

Application	What s, t, edges and capacities stand for
Network routing / bandwidth	Routers are vertices, links carry Mbps capacities; max flow = the most traffic s can send to t; the min cut is the set of links whose failure disconnects them cheapest.
Transportation & logistics	Depots, roads with truck limits, warehouses — how many tonnes per day can reach the port.
Image segmentation	Pixels are vertices; s = "foreground", t = "background"; edge weights = similarity. The **min cut** is the cheapest boundary between object and background (graph cuts).
Disjoint paths (Menger)	Unit capacities: max flow = number of **edge-disjoint** s–t paths = fewest links whose removal disconnects s from t.
Scheduling / assignment	Crews to flights, students to projects — matching with capacities; also project selection and baseball elimination.

Exam trap Max flow equals the **minimum** cut, not "the sum of all capacities" nor "the largest edge". A flow network can have several minimum cuts but only one max-flow value.

Memory trick "**A chain is as strong as its weakest link — a network is as wide as its narrowest cut.**"

5 Approximation Algorithms: the Ratio ρ, and Vertex Cover

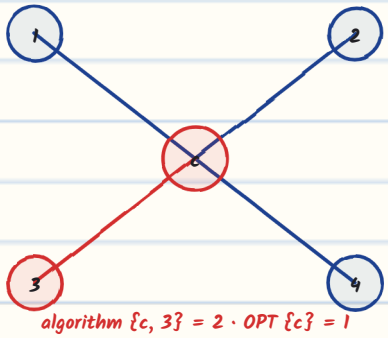
provably close, provably fast ★

Definition An **approximation algorithm** runs in polynomial time on an **NP-hard optimisation problem** and returns a solution with a **proven guarantee** of how far it can be from the optimum. (Why the exact answer is out of reach — NP-hardness — is the subject of the next lesson.) A heuristic with no guarantee is not an approximation algorithm.

Problem type	Approximation ratio ρ(n)	Reading it
Minimisation (cover, tour)	$\rho = C / C^* \geq 1$	C = algorithm's cost, C^* = optimal. ρ = 2: never more than twice the optimum.
Maximisation (knapsack, cut)	$\rho = C^* / C \geq 1$	ρ = 2: never less than half the optimum.
Combined (CLRS)	$\max(C/C^*, C^*/C) \leq \rho(n)$	ρ = 1 means exact; the nearer to 1, the better.

Problem	Algorithm	Ratio	Note
Vertex cover	take both ends of a maximal matching	**2**	sheet below
Metric TSP	MST, preorder walk, shortcut	**2**	Christofides **1.5**
Set cover	greedy: most uncovered elements	**$H(n) \approx \ln n$**	$\leq \ln n + 1$
0/1 knapsack	value-scaling DP	**$1 + \varepsilon$**	FPTAS

```
APPROX-VERTEX-COVER(G)
  C = ∅;  E' = E
  while E' ≠ ∅:
    pick any edge (u,v) in E'
    C = C ∪ {u, v}
    delete from E' every edge touching u or v
  return C                    // O(V + E)
```



algorithm {c, 3} = 2 · OPT {c} = 1

a star: the bound 2 is tight

Why ratio 2 ✍ proof The picked edges form a **matching M** (no two share a vertex), and $|C| = 2|M|$. Any cover — the optimal C^* too — must contain at least one endpoint of each edge of M, and those endpoints are all different, so $|C^*| \geq |M|$. Hence $|C| = 2|M| \leq 2|C^*|$.

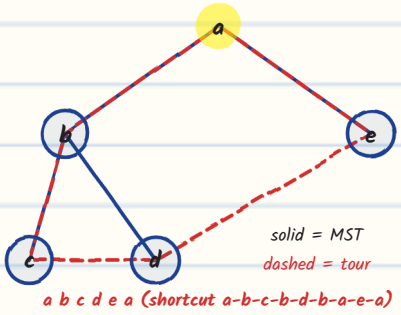
Exam trap ✍ two **1.** The "pick the highest-degree vertex" greedy is **not** a 2-approximation — its ratio grows like log n. The 2 belongs to the **both-endpoints** rule. **2.** A minimisation ratio of 2 allows up to 2 × OPT, never below OPT.

Memory trick "**Grab the whole edge, pay double at most.**" One endpoint of each grabbed edge was needed anyway; the other is the price of not knowing which.

6 Metric TSP, Set Cover, Knapsack — and PTAS vs FPTAS

the classic ratios ★

Metric TSP: the MST tour Distances obey the triangle inequality $d(u,w) \le d(u,v) + d(v,w)$. **1.** Build an **MST** T (Prim/Kruskal, previous lesson). **2.** Walk it in **DFS preorder**: the full walk uses each MST edge twice, cost $2 \cdot w(T)$. **3. Shortcut** repeated vertices — by the triangle inequality this never adds cost. Since deleting one edge of the optimal tour leaves a spanning tree, $w(T) \le OPT$, so tour $\le 2 \cdot w(T) \le 2 \cdot OPT$.



preorder a, b, c, d, e, then back to a

Beyond 2 — Christofides (1976) MST + a **minimum-weight perfect matching** on the odd-degree MST vertices gives an Eulerian graph; shortcut its Euler tour → **ratio 1.5** for metric TSP. **General TSP** (no triangle inequality) has no constant-ratio approximation unless $P = NP$.

Set cover: greedy Universe U of n elements, a family of subsets; choose the fewest subsets covering U. Greedy: repeatedly take the set covering the most still-uncovered elements. Ratio $H(n) = 1 + \frac{1}{2} + \dots + \frac{1}{n} \le \ln n + 1$ — essentially the best possible in polynomial time (unless $P = NP$).

Worked example $U = \{1,\dots,6\}$; $S1 = \{1,2,3,4\}$, $S2 = \{4,5\}$, $S3 = \{5,6\}$, $S4 = \{1,6\}$. Greedy takes S1 (4 new), then S3 (5, 6: 2 new) → **2 sets**, all covered. Here greedy is optimal; on adversarial inputs it can be $\approx \ln n$ times worse.

0/1 knapsack: FPTAS The exact DP is $O(nW)$ — **pseudo-polynomial**. Scale every value down: $v' = \lfloor v / K \rfloor$ with $K = \varepsilon \cdot v_{max} / n$, run the DP indexed by value in $O(n^3/\varepsilon)$, and the answer is $\ge (1 - \varepsilon) \cdot OPT$. Simpler: max(greedy by value/weight, best single item) is a **2-approximation**. (Fractional knapsack needs none of this — greedy is exact.)

Scheme	Guarantee	Running time	Example
PTAS	within $(1 + \varepsilon)$ for **every fixed** $\varepsilon > 0$	polynomial in n, may be exponential in $1/\varepsilon$, e.g. $O(n^{(1/\varepsilon)})$	Euclidean TSP, bin-packing variants
FPTAS	within $(1 + \varepsilon)$	**polynomial in n and $1/\varepsilon$**, e.g. $O(n^3/\varepsilon)$	0/1 knapsack, subset sum

Exam trap ✍ three **1.** Every FPTAS is a PTAS, **not** the reverse. **2.** A strongly NP-hard problem has no FPTAS unless $P = NP$; knapsack is only *weakly* NP-hard. **3.** "Set cover greedy ratio 2" is wrong — it is **ln n**.

Memory trick "**Cover 2, Tour 2 then 1.5, Set ln n, Knapsack 1+ε.**" And "**FPTAS** is **Fully** polynomial — in **both** n and $1/\varepsilon$."

7 Parallel Algorithms: Speedup, Efficiency, Granularity, Amdahl's Law

the formula sheet ★

Definition A **parallel algorithm** divides a problem into sub-problems solved simultaneously on several processors, then combines the results. Its quality is measured against the **best sequential** algorithm, not against itself on one processor.

Measure	Formula	Meaning
Speedup S	$S = T_1 / T_p$	T_1 = best sequential time, T_p = time on p processors. Ideal (linear): $S = p$.
Efficiency E	$E = S / p$	Fraction of processor time doing useful work; $0 < E \leq 1$ normally.
Cost	$p \times T_p$	**Cost-optimal** if $p \cdot T_p = \Theta(T_1)$.
Granularity	computation / communication	**Fine**-grained: many small tasks, much communication. **Coarse**-grained: few large tasks.
Work-span	$T_p \geq T_1/p$, $T_p \geq T\infty$	$T\infty$ (span) = longest dependency chain; parallelism = $T_1/T\infty$; Brent: $T_p \leq T_1/p + T\infty$.

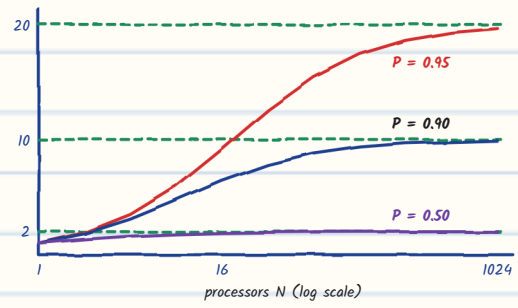
Data vs task parallelism **Data**: the same operation on different pieces of data — vector addition, matrix multiply, image filters (SIMD, GPUs). **Task**: different operations at once — pipeline stages, a web server's independent requests (MIMD).

Flynn's taxonomy **SISD** classic uniprocessor · **SIMD** vector units, GPUs · **MISD** rare (fault-tolerant pipelines) · **MIMD** multicores, clusters. PRAM models: **EREW**, **CREW**, **CRCW** (exclusive/concurrent read/write).

Amdahl's law (fixed problem size) If a fraction **P** of the work can run in parallel on **N** processors and $1 - P$ stays serial: $S(N) = 1 / ((1 - P) + P/N)$. As $N \to \infty$, $S \to 1 / (1 - P)$ — the serial part caps the speedup.

Worked example $P = 0.9$, $N = 10$: $S = 1 / (0.1 + 0.09) = 1/0.19 \approx 5.26$; $E = 5.26/10 \approx 53\%$; limit $1/0.1 = 10$, however many processors. $P = 0.8$, $N = 4$: $S = 1/(0.2 + 0.2) = 2.5$; limit 5.

Gustafson's law (scaled size) When the problem grows with N: $S = (1 - P) + P \cdot N$ (P = parallel share of the parallel run). $P = 0.9$, $N = 10 \to 0.1 + 9 = 9.1$. Amdahl is pessimistic, Gustafson optimistic.



speedup flattens at $1/(1-P)$: 2, 10, 20

Exam trap In Amdahl's law **P is the parallel fraction**; mixing it with the serial fraction inverts every answer. Speedup **above p** (superlinear) can occur through cache effects, but is not the norm.

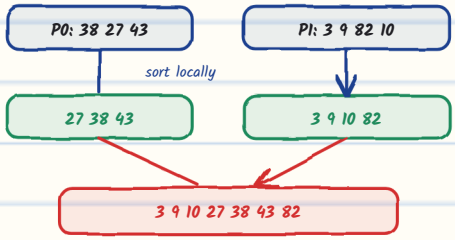
Memory trick "**The slowest part sets the pace.**" Ten percent serial ⇒ never more than ten times faster.

8 Parallel Sorting, Matrix Multiplication, BFS — and the Pitfalls

the standard algorithms ★

Problem	Parallel approach	Cost / time
Reduction (sum of n)	Pairwise tree: n/2 adds, then n/4, ...	$O(\log n)$ steps, n/2 processors
Parallel merge sort	Split among p processors, sort locally, merge in a tree of log p levels.	$O((n/p)\log(n/p))$ local sort + merging
Bitonic sort	Sorting **network**: build bitonic sequences, then bitonic-merge with compare-exchange; n a power of 2.	$O(\log^2 n)$ time, $O(n \log^2 n)$ comparisons
Matrix multiply	**Block** decomposition; **Cannon's** algorithm on a $\sqrt{p} \times \sqrt{p}$ processor grid.	$O(n^3/p)$ compute, $\sqrt{p}$ shift steps
Graph search	**Level-synchronous BFS**: expand the whole frontier in parallel; also parallel Dijkstra (Δ-stepping).	rounds = depth of the BFS tree

Worked example: parallel merge sort Array [38, 27, 43, 3, 9, 82, 10] on 2 processors. **P0** gets [38, 27, 43] → sorts to [27, 38, 43]; **P1** gets [3, 9, 82, 10] → [3, 9, 10, 82] — both at the same time. One merge step → **[3, 9, 10, 27, 38, 43, 82]**. With p processors the merge tree has $\log_2 p$ levels.



split, sort in parallel, merge

Bitonic half-cleaner A **bitonic** sequence rises then falls (or a rotation of one). Compare item i with item i + n/2 and swap if out of order: [2, 5, 8, 9 | 7, 6, 3, 1] → low [2, 5, 3, 1], high [7, 6, 8, 9]. Every low ≤ every high, and each half is bitonic again — recurse. n = 8 needs 3·4/2 = **6** compare stages.

Cannon's algorithm Processor P(i,j) holds blocks A(i,j), B(i,j). **Skew**: shift row i of A left by i, column j of B up by j. Then $\sqrt{p}$ times: **multiply-add** local blocks, shift A one left and B one up. Each processor stores only one block of each matrix at a time.

Challenge	What goes wrong / the remedy
Communication overhead	Sending data costs time; fine granularity can make T_p worse than T_1.
Load balancing	Uneven pieces leave processors idle — dynamic scheduling, work stealing.
Synchronisation, races, deadlocks	Barriers make fast processors wait; unordered updates race (BFS "visited" flag → atomic compare-and-swap); circular lock waits deadlock.

Applications Scientific simulation (weather, physics) · **big data**: Hadoop MapReduce, Apache Spark · graphics **rendering** on GPUs · **distributed ML** training (data-parallel gradients, all-reduce).

Exam trap Bitonic sort does **more** comparisons than merge sort ($n \log^2 n$ vs $n \log n$) — it wins because its compare pattern is fixed, ideal for hardware and GPUs.

Night-Before-the-Exam Sheet

Fact sheet

$0 \le f(u,v) \le c(u,v)$
$|f|$ = net flow out of s
Augmenting path: $s \to t$ in G_f
FF: $O(E \cdot |f^*|)$, integer caps
Edmonds–Karp: BFS, $O(VE^2)$
Dinic unit caps: $E \cdot \min(V^{2/3}, E^{1/2})$
max flow = min cut
Matching = unit-cap flow
Min: $\rho = C/C^* \ge 1$
Vertex cover: 2
Christofides: 1.5
Set cover: $H(n) \le \ln n + 1$
PTAS: poly in n
$S = T_1/T_p$ · $E = S/p$
Amdahl $1/((1-P)+P/N)$
Gustafson $(1-P)+P \cdot N$

In = out, except s and t
Residual fwd $c - f$, back f
Bottleneck = min c_f on path
Irrational caps: may not stop
Dinic: $O(V^2E)$
Push–relabel $O(V^2E)$, FIFO $O(V^3)$
Cut counts $S \to T$ edges only
Hopcroft–Karp $O(E\sqrt{N})$
Max: $\rho = C^*/C \ge 1$
Metric TSP: 2 (MST)
General TSP: no constant
Knapsack: FPTAS $O(n^3/\varepsilon)$
FPTAS: poly in n and $1/\varepsilon$
Cost = $p \cdot T_p$
Limit $1/(1-P)$
Bitonic $O(\log^2 n)$

→ Top 12 one-liners for MCQs

1. Max flow = capacity of a **min** cut
2. Edmonds–Karp = FF + **BFS**
3. E-K **VE^2**, Dinic **V^2E**
4. Back edges let flow be **cancelled**
5. Conservation fails only at s, t
6. Matching: all capacities **1**
7. Vertex cover: both endpoints, **2**
8. MST tour **2**, Christofides **1.5**
9. Greedy set cover **ln n**
10. FPTAS ⊂ PTAS
11. $P = 0.9 \Rightarrow S \le 10$
12. $E = S/p$, ideal $E = 1$

Top traps quoting Dinic's **unit-capacity** bound as general · FF **always terminates** (only with integer/rational caps) · adding **T→S** edges to a cut · max-degree greedy as **2-approx** · set cover ratio **2** · **FPTAS** exponential in $1/\varepsilon$ · Amdahl with the **serial** fraction as P.

All tricks together Forward = room left, backward = regret · Ford any road, E-K shortest, Dinic levels, push–relabel downhill · As wide as the narrowest cut · Grab the whole edge, pay double · Cover 2, Tour 2 → 1.5, Set ln n, Knapsack 1+ε · The slowest part sets the pace.



how much · how close · how fast

three numbers, three toolkits

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. By the max-flow min-cut theorem, the maximum s–t flow equals:
(a) The number of edges (b) The sum of all capacities
(c) The capacity of a minimum s–t cut (d) The largest edge capacity

2. Edmonds–Karp (FF with BFS augmenting paths) runs in:
(a) $O(VE^2)$ (b) $O(E \log V)$
(c) $O(V^2)$ (d) $O(E \cdot \lvert f^* \rvert)$ only

3. An edge with capacity 10 carries flow 6. Its residual edges are:
(a) forward 6, backward 4 (b) forward 10, backward 0
(c) forward 16, backward 6 (d) forward 4, backward 6

4. Taking both endpoints of a maximal matching gives a vertex cover within a factor of:
(a) 1.5 (b) 2
(c) log n (d) n

5. The MST-based (double-tree) algorithm for **metric** TSP has ratio:
(a) 2 (b) 1.5
(c) ln n (d) no constant

6. An FPTAS runs in time polynomial in:
(a) n only (b) $1/\varepsilon$ only
(c) both n and $1/\varepsilon$ (d) neither

7. By Amdahl's law, with 90% parallel code on 10 processors the speedup is about:
(a) 9.1 (b) 5.26
(c) 10 (d) 4.75

8. A program gets speedup 6 on 8 processors. Its efficiency is:
(a) 1.33 (b) 48
(c) 0.60 (d) 0.75

9. Bitonic sort with n/2 comparators per stage takes parallel time:
(a) $O(\log n)$ (b) $O(n)$
(c) $O(\log^2 n)$ (d) $O(n \log n)$

10. The greedy set-cover algorithm has approximation ratio:
(a) $H(n) \approx \ln n$ (b) 2
(c) $1 + \varepsilon$ (d) 1.5

Answer key → 1-c · 2-a · 3-d · 4-b · 5-a · 6-c · 7-b · 8-d · 9-c · 10-a

Written question Define a **flow network** and state the capacity and conservation constraints. Find the maximum flow of the network on sheet 2 by Ford–Fulkerson with BFS paths, and verify it with a **minimum cut**. State **Amdahl's law** and compute the speedup for P = 0.8 on 4 processors and its upper limit. 10 marks

1. **Definition (2)** — directed graph, capacity $c(u,v) \geq 0$ on each edge, source s, sink t. **Capacity:** $0 \leq f(u,v) \leq c(u,v)$. **Conservation:** inflow = outflow at every vertex except s and t. 2
2. **Augmentations (3)** — $s \to A \to B \to t$ (4), $s \to A \to D \to t$ (6), $s \to C \to D \to t$ (4), $s \to C \to D \to B \to t$ (5): total **19**; then no $s \to t$ path exists in the residual graph. 3
3. **Min cut (2)** — $S = \{s, C\}$ (reachable from s in G_f), T = rest; $S \to T$ edges $s \to A$ (10) + $C \to D$ (9) = **19** = $\lvert f \rvert$, so the flow is maximum by the max-flow min-cut theorem. 2
4. **Amdahl (3)** — $S = 1/((1 - P) + P/N) = 1/(0.2 + 0.8/4) = 1/0.4 = \mathbf{2.5}$; efficiency 2.5/4 = 62.5%; limit $N \to \infty$: $1/(1 - 0.8) = \mathbf{5}$ — the 20% serial part caps it. 3

→ Before the exam ✓

- ✓ Can run Ford–Fulkerson by hand with residuals
- ✓ Can find the min cut from the final residual graph
- ✓ Can quote the four max-flow complexities
- ✓ Can prove the vertex-cover ratio 2
- ✓ Can separate PTAS from FPTAS
- ✓ Can compute Amdahl speedup, limit and efficiency